Homework 4: Reinforcement Learning

111550170 卓建均

Part I. Implementation:

Part 1. Q learning in Taxi-v3

```python
def choose_action(self, state):
    """
    Choose the best action with given state and epsilon.

    Parameters:
        state: A representation of the current state of the enviornment.
        epsilon: Determines the explore/expliot rate of the agent.

    Returns:
        action: The action to be evaluated.
    """
    # Begin your code
    """
    Fisrt, I generate a ramdom number in [0,1] to determine to do exploitation
    and return the index of the minimum Q of the given state in Q-table
    or exploration and return random action
    """
    if (np.random.uniform(0, 1) > self.epsilon):
        return np.argmax(self.qtable[state])
    else:
        return self.env.action_space.sample()
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

```python
def learn(self, state, action, reward, next_state, done):
    """
    Calculate the new q-value base on the reward and state transformation observered after taking the action.

    Parameters:
        state: The state of the enviornment before taking the action.
        action: The exacuted action.
        reward: Obtained from the enviornment after taking the action.
        next_state: The state of the enviornment after taking the action.
        done: A boolean indicates whether the episode is done.

    Returns:
        None (Don't need to return anything)
    """
    # # Begin your code
    """
    When the episode is not finished, the reward is updated by adding the discounted maximum
    Q-value of the next state. Then, the target value for the Q-value update is calculated.
    Finally, the Q-value for the current state-action pair is updated using the learning
    rate and the target value. If the episode is finished, the updated Q-table is saved to disk.
    """
    if not done:
        reward += self.gamma * np.max(self.qtable[next_state])
    target = reward * self.learning_rate
    self.qtable[state, action] = (1-self.learning_rate)*self.qtable[state,action] + target
    if done:
        np.save("./Tables/taxi_table.npy", self.qtable)
    # #raise NotImplementedError("Not implemented yet.")
    # # End your code
```

```python
def check_max_Q(self, state):
    """
    - Implement the function calculating the max Q value of given state.
    - Check the max Q value of initial state

    Parameter:
        state: the state to be check.
    Return:
        max_q: the max Q value of given state
    """
    # Begin your code
    """
    This function calculates the maximum Q-value for a given state by finding the
    maximum value in the Q-table for that state. Then, it returns this maximum Q-value.
    """
    max_q = np.max(self.qtable[state])
    return max_q
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

Part 2. Q learning in CartPole-v0

```python
def init_bins(self, lower_bound, upper_bound, num_bins):
    """
    Slice the interval into #num_bins parts.
    Parameters:
        lower_bound: The lower bound of the interval.
        upper_bound: The upper bound of the interval.
        num_bins: Number of parts to be sliced.
    Returns:
        a numpy array of #num_bins - 1 quantiles.
    Example:
        Let's say that we want to slice [0, 10] into five parts,
        that means we need 4 quantiles that divide [0, 10].
        Thus the return of init_bins(0, 10, 5) should be [2. 4. 6. 8.].
    Hints:
        1. This can be done with a numpy function.
    """
    # Begin your code
    """
    This function slices the interval by lower_bound and upper_bound into num_bins
    parts and returns an array of quantiles. It uses NumPy's linspace function to
    divide the interval, excluding the upper bound, and then selects all elements
    except the first one, as the first element is essentially the lower bound.
    """
    return np.linspace(lower_bound, upper_bound, num_bins, endpoint=False)[1:]
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

```python
def discretize_value(self, value, bins):
    """
    Discretize the value with given bins.
    Parameters:
        value: The value to be discretized.
        bins: A numpy array of quantiles
    returns:
        The discretized value.
    Example:
        With given bins [2. 4. 6. 8.] and "5" being the value we're going to discretize.
        The return value of discretize_value(5, [2. 4. 6. 8.]) should be 2, since 4 <= 5 < 6 where [4, 6) is the 3rd bin.
    Hints:
        1. This can be done with a numpy function.
    """
    # Begin your code
    """
    This function discretizes a given value using the provided bins array.
    It uses NumPy's digitize function, which returns the indices of the bins
    to which each value in the input array belongs.
    """
    return np.digitize(value, bins)
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

```python
def discretize_observation(self, observation):
    """
    Discretize the observation which we observed from a continuous state space.
    Parameters:
        observation: The observation to be discretized, which is a list of 4 features:
            1. cart position.
            2. cart velocity.
            3. pole angle.
            4. tip velocity.
    Returns:
        state: A list of 4 discretized features which represents the state.
    Hints:
        1. All 4 features are in continuous space.
        2. You need to implement discretize_value() and init_bins() first
        3. You might find something useful in Agent.__init__()
    """
    # Begin your code
    """
    This function discretizes a continuous observation into a discrete state representation.
    It iterates over each feature of the observation and discretizes it using the corresponding
    pre-defined bins. Then return a tuple containing the discretized values of all four features,
    representing the discretized state.
    """
    discretized_state = []
    for obs, bin in zip(observation, self.bins):
        discretized_value = self.discretize_value(obs, bin)
        discretized_state.append(discretized_value)
    return tuple(discretized_state)
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

```python
def choose_action(self, state):
    """
    Choose the best action with given state and epsilon.
    Parameters:
        state: A representation of the current state of the enviornment.
        epsilon: Determines the explore/expliot rate of the agent.
    Returns:
        action: The action to be evaluated.
    """
    # Begin your code
    """
    I randomly select a number between [0,1], and if this number is greater
    than epsilon, it chooses the action with the highest Q-value for exploitation.
    Otherwise, it selects a random action exploration.
    """
    if (np.random.uniform(0, 1) > self.epsilon):
        return np.argmax(self.qtable[state])
    else:
        return self.env.action_space.sample()
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

```python
def learn(self, state, action, reward, next_state, done):
    """
    Calculate the new q-value base on the reward and state transformation observered after taking the action.
    Parameters:
        state: The state of the enviornment before taking the action.
        action: The exacuted action.
        reward: Obtained from the enviornment after taking the action.
        next_state: The state of the enviornment after taking the action.
        done: A boolean indicates whether the episode is done.
    Returns:
        None (Don't need to return anything)
    """
    # Begin your code
    """
    This function implements the Q-learning algorithm to update the Q-values based on the reward obtained from
    the environment and the observed state transitions. It calculates the maximum Q-value for the next state,
    and if the episode is done, it sets the maximum next Q-value to 0. Then, it updates the Q-value for the current
    state-action pair using the Q-learning formula. Finally, if the episode is done, it saves the Q-table to a file.
    """
    max_nxt = np.max(self.qtable[next_state])
    if done:
        max_nxt = 0
    self.qtable[state + (action,)] = (1 - self.learning_rate) * self.qtable[state + (action,)] +self.learning_rate * (reward + self.gamma * max_nxt)
    if done:
        np.save("./Tables/cartpole_table.npy", self.qtable)
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

```python
def check_max_Q(self):
    """
    - Implement the function calculating the max Q value of initial state(self.env.reset()).
    - Check the max Q value of initial state
    Parameter:
        self: the agent itself.
        (Don't pass additional parameters to the function.)
        (All you need have been initialized in the constructor.)
    Return:
        max_q: the max Q value of initial state(self.env.reset())
    """
    # Begin your code
    """
    This function calculates the maximum Q-value for the initial state of the environment. It resets
    the environment using self.env.reset() to get the initial observation, then discretizes it to
    obtain the initial state. After that, it retrieves the Q-values corresponding to the initial state
    from the Q-table and calculates the maximum Q-value. Finally, it returns this maximum Q-value.
    """
    initial_state = tuple(self.discretize_observation(self.env.reset()))
    max_q = np.max(self.qtable[initial_state])
    return max_q
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

Part 3. DQN in CartPole-v0

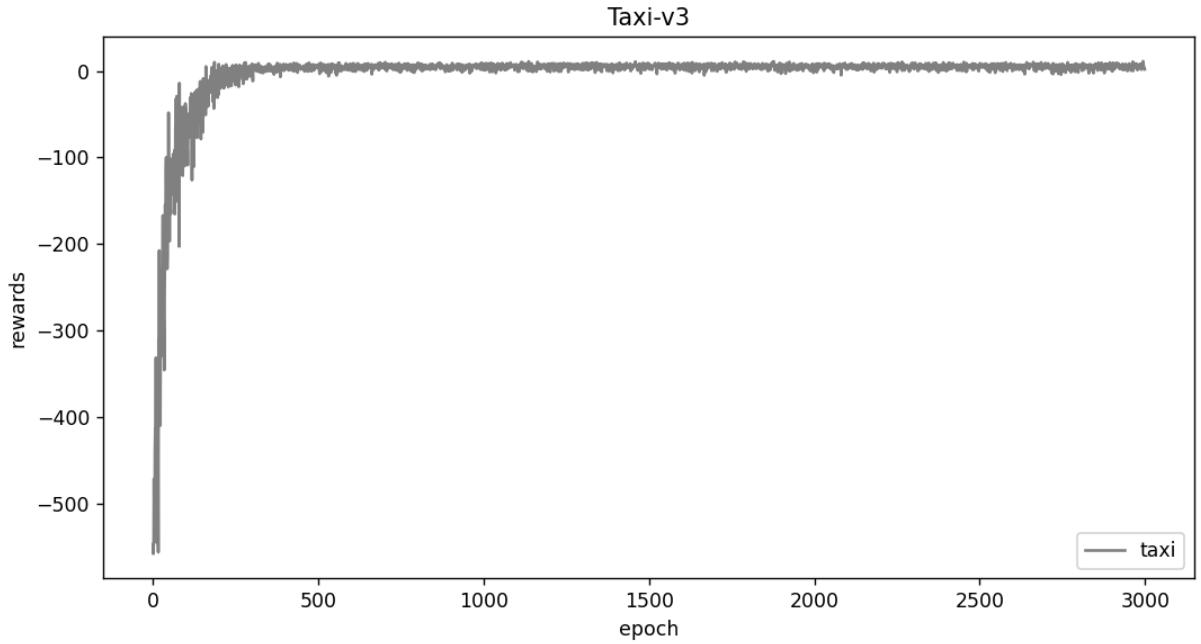
```python
def learn(self):
    3. Forward the data to the evaluate net and the target net.
    4. Compute the loss with MSE.
    5. Zero-out the gradients.
    6. Backpropagation.
    7. Optimize the loss function.
    -----
    Parameters:
        self: the agent itself.
        (Don't pass additional parameters to the function.)
        (All you need have been initialized in the constructor.)
    Returns:
        None (Don't need to return anything)
    '''
    if self.count % 100 == 0:
        self.target_net.load_state_dict(self.evaluate_net.state_dict())
    # Begin your code
    """
    Just follow the hints to implement the learning function from step 1 to step 7.
    Additionally, it initializes the self.step attribute if it's not already
    initialized and saves the target network's state every 100 steps.
    """
    if not hasattr(self, "step"):
        self.step = 0
    batch_state, batch_action, batch_reward, batch_next_state, batch_done = self.buffer.sample(self.batch_size)
    action=torch.tensor(np.asarray(batch_action).reshape(len(batch_action), 1), dtype = torch.long)
    reward=torch.tensor(np.asarray(batch_reward).reshape(len(batch_reward), 1), dtype = torch.float)
    q_evaluate = self.evaluate_net(torch.tensor(np.asarray(batch_state), dtype = torch.float)).gather(1, action)
    q_next = self.target_net(torch.tensor(np.asarray(batch_next_state), dtype = torch.float)).detach()
    q_target = reward + self.gamma * q_next.max(1).values.unsqueeze(-1)
    for i in range(len(batch_done)):
        if batch_done[i]:
            q_target[i][0] = 0
    loss_function = nn.MSELoss()
    loss = loss_function(q_evaluate, q_target)
    self.optimizer.zero_grad()
    loss.backward()
    self.optimizer.step()
    self.count += 1
    self.step += 1
    if self.step % 100 == 0:
        torch.save(self.target_net.state_dict(), "./Tables/DQN.pt")
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

```python
def choose_action(self, state):
    """
    - Implement the action-choosing function.
    - Choose the best action with given state and epsilon
    Parameters:
        self: the agent itself.
        state: the current state of the enviornment.
        (Don't pass additional parameters to the function.)
        (All you need have been initialized in the constructor.)
    Returns:
        action: the chosen action.
    """
    with torch.no_grad():
        # Begin your code
        """
        First it disables gradient tracking using torch.no_grad() since no gradients are needed for
        action selection. Then, I convert the state to a PyTorch tensor. I choose the action based on
        an epsilon-greedy strategy: with probability (1 - epsilon), it exploits the current policy by
        selecting the action with the highest Q-value according to the evaluation network; otherwise,
        with probability epsilon, it explores by choosing a random action from the environment's action
        space. Finally, it returns the chosen action.
        """
        x = torch.tensor(state, dtype=torch.float)
        if np.random.uniform(0, 1) > self.epsilon:
            action_values = self.evaluate_net(x)
            action = torch.argmax(action_values).item()
        else:
            action = env.action_space.sample()
        #raise NotImplementedError("Not implemented yet.")
        # End your code
    return action
```

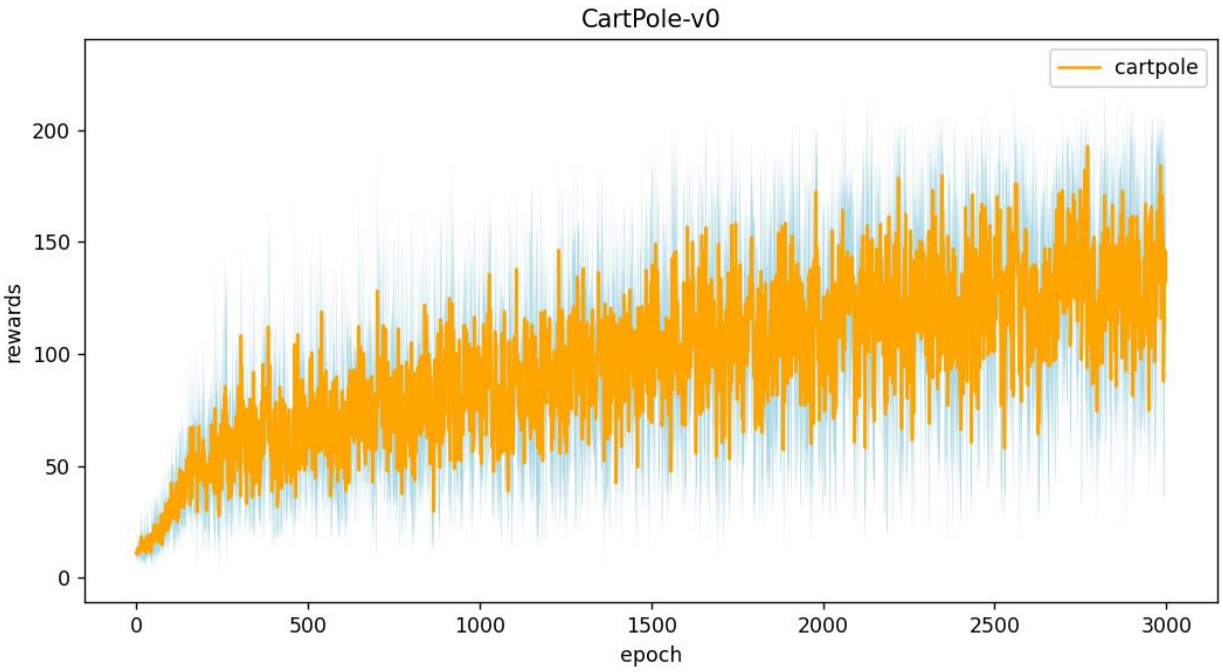
```python
def check_max_Q(self):
    """
    - Implement the function calculating the max Q value of initial state(self.env.reset()).
    - Check the max Q value of initial state
    Parameter:
        self: the agent itself.
        (Don't pass additional parameters to the function.)
        (All you need have been initialized in the constructor.)
    Return:
        max_q: the max Q value of initial state(self.env.reset())
    """
    # Begin your code
    """
    I retrieve the initial state using self.env.reset() and converts it to a PyTorch tensor.
    Then, it forwards this tensor through the target network to obtain the Q-values for the
    initial state. After that, it finds the maximum Q-value and converts it to a float.
    Finally, it returns the maximum Q-value.
    """
    initial_state = torch.tensor(self.env.reset(), dtype = torch.float)
    q_value = self.target_net(initial_state).detach()
    max_q = q_value.max(0).values.unsqueeze(-1)
    max_q = float(max_q[0])
    return max_q
    #raise NotImplementedError("Not implemented yet.")
    # End your code
```

Part II. Experiment Results:

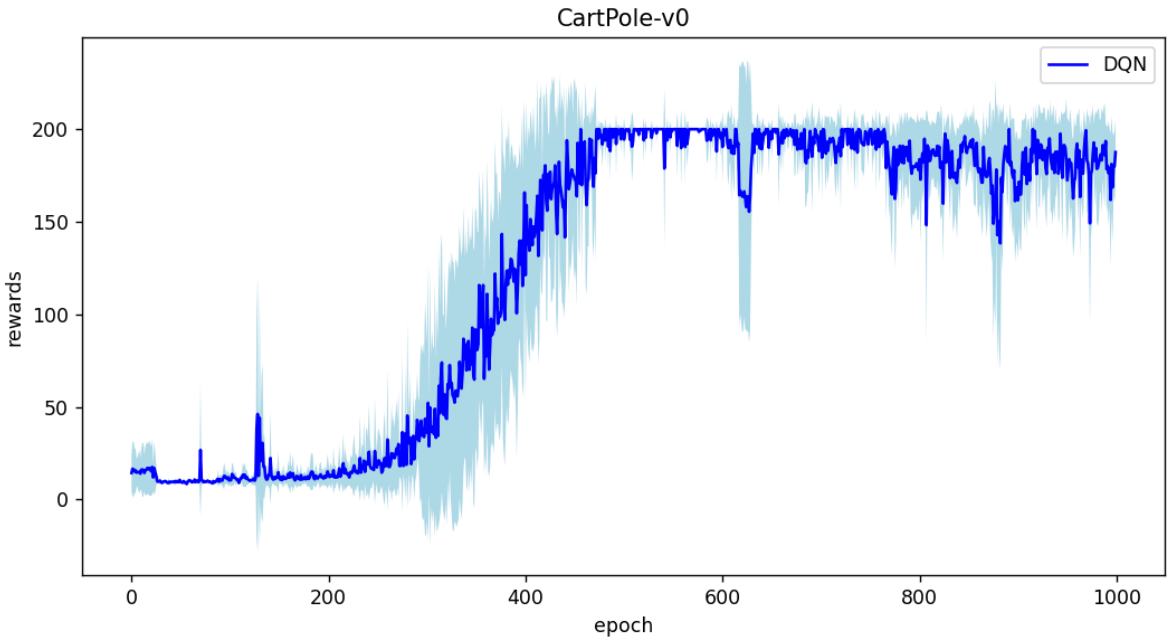
1. taxi.png



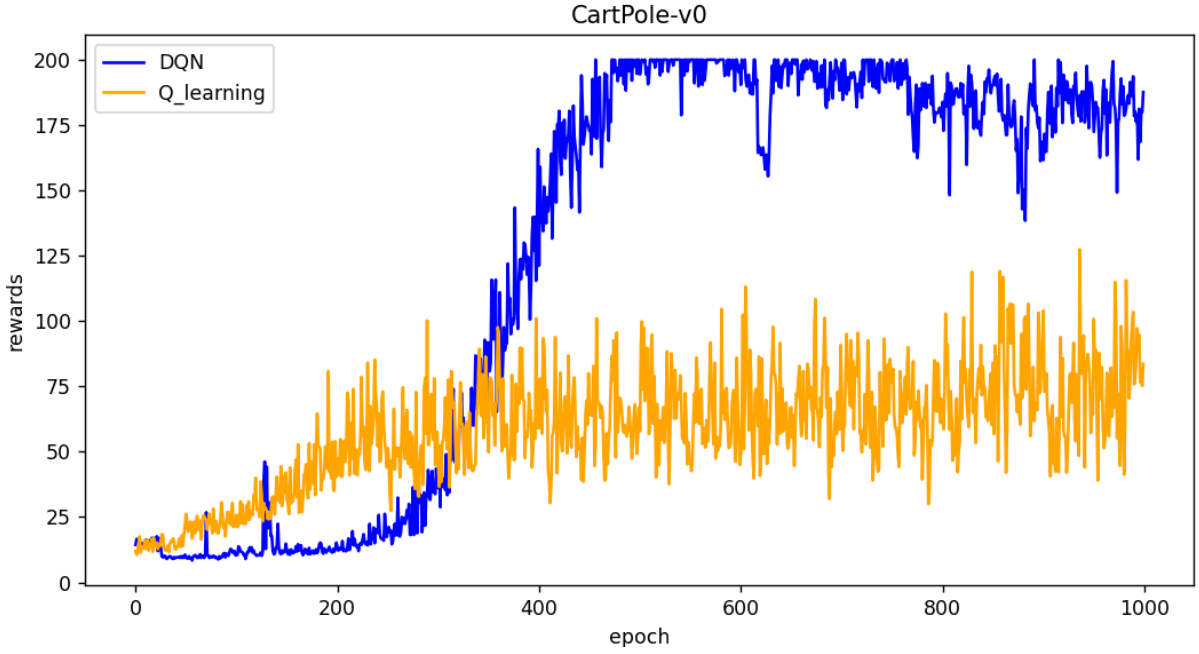
2. cartpole.png



3. DQN.png



4. compare.png



Part III. Question Answering:

1. Calculate the optimal Q-value of a given state in Taxi-v3, and compare with the Q-value you learned (Please screenshot the result of the "check_max_Q" function to show the Q-value you learned).

A: Due to taxi at (2, 2), passenger at Y, and destination at R, the step would be left >left->down ->down->pick->up->up->up->up->drop. Therefore, the rewards are 9 steps of-1 and 1 step of 20. Optimal Q-value: −1× (1−γ9) / (1−γ) + 20×(γ9) = −1× (1−0.9^9) / (1−0.9) + 20×(0.9^9) = 1.622614... , which is close to the max Q.

```
PS C:\Users\choan\Desktop\intro_ai\4\AI_HW4> python taxi.py
#1 training progress
100%|| 3000/3000 [00:02<00:00, 1318.68it/s]
#2 training progress
100%|| 3000/3000 [00:02<00:00, 1322.61it/s]
#3 training progress
100%|| 3000/3000 [00:02<00:00, 1345.16it/s]
#4 training progress
100%|| 3000/3000 [00:02<00:00, 1183.47it/s]
#5 training progress
100%|| 3000/3000 [00:02<00:00, 1302.08it/s]
average reward: 7.86
Initail state:
taxi at (2, 2), passenger at Y, destination at R
max Q:1.6226146699999995
```

2. Calculate the optimal Q-value of the initial state in CartPole-v0, and compare with the Q-value you learned(both cartpole.py and DQN.py). (Please screenshot the result of the "check_max_Q" function to show the Q-value you learned)

A: Optimal Q-value: (1−γaverage-reward) / (1−γ) ≈ 1 / (1−0.97) = 33.333... , which is close to the max Q of Q-learning and DQN, yet DQN is more closer.

```
PS C:\Users\choan\Desktop\intro_ai\4\AI_HW4> python cartpole.py
#1 training progress
100%|| 3000/3000 [00:18<00:00, 163.67it/s]
#2 training progress
100%|| 3000/3000 [00:14<00:00, 210.32it/s]
#3 training progress
100%|| 3000/3000 [00:12<00:00, 233.28it/s]
#4 training progress
100%|| 3000/3000 [00:15<00:00, 187.57it/s]
#5 training progress
100%|| 3000/3000 [00:14<00:00, 202.00it/s]
average reward: 167.88
max Q:30.53191246757374
```

```
PS C:\Users\choan\Desktop\intro_ai\4\AI_HW4> python DQN.py
#1 training progress
100%|| 1000/1000 [02:05<00:00,  7.99it/s]
#2 training progress
100%|| 1000/1000 [03:11<00:00,  5.23it/s]
#3 training progress
100%|| 1000/1000 [03:13<00:00,  5.16it/s]
#4 training progress
100%|| 1000/1000 [03:09<00:00,  5.28it/s]
#5 training progress
100%|| 1000/1000 [03:47<00:00,  4.40it/s]
reward: 155.14
max Q:34.06072235107422
```

3. a. Why do we need to discretize the observation in Part 2?

A: Because the interval is continuous, which is not easy to determine the state of the CartPole. As a result, we need to discretize it to get the state.

3. b. How do you expect the performance will be if we increase "num_bins"?

A: I think the performance will become better if we increase the num_bins, which represents the number of the states in the bounded interval. Because when we increase the number of the

states, it implies that we have more states to approximate the continuous interval, which leads to the better performance.

3. c. Is there any concern if we increase “num_bins”?

A: Increasing "num_bins" can result in concerns such as longer time required to update and save the Q table due to the increased number of states, as well as the increased memory required to save the larger Q table.

4. Which model (DQN, discretized Q learning) performs better in Cartpole-v0, and what are the reasons?

A: DQN outperforms discretized Q learning in the CartPole-v0 environment. The reason is that Q learning discretizes the continuous data into states, which can result in data loss. In contrast, DQN can use the continuous data and preserve more details, leading to better performance.

5. a. What is the purpose of using the epsilon greedy algorithm while choosing an action?

A: The epsilon-greedy algorithm serves the purpose of balancing exploration and exploitation in action selection. It allows for the selection of the best-known action while also exploring new options, thus taking advantage of prior knowledge and discovering new possibilities.

5. b. What will happen, if we don’t use the epsilon greedy algorithm in the CartPole-v0 environment?

A: If the epsilon-greedy algorithm is not used in the CartPole-v0 environment, there are two potential scenarios. If only exploration is used, there will be no way to choose the best-known action, as all actions will be random. If only exploitation is used, the algorithm can only rely on the known information and may miss unknown high-performance conditions without any randomness or exploration.

5. c. Is it possible to achieve the same performance without the epsilon greedy algorithm in the CartPole-v0 environment? Why or Why not?

A: It may be possible to achieve the same performance without the epsilon-greedy algorithm in the CartPole-v0 environment if we can find another method to replace the learning rate in the algorithm while maintaining the same proportion of exploration/exploitation. With the same distribution, there is a possibility to achieve the same performance.

5. d. Why don’t we need the epsilon greedy algorithm during the testing section?

A: The epsilon-greedy algorithm is not needed during the testing section because it is used for exploration and exploitation during training. In the testing section, the goal is simply to find the best path and obtain the best reward, and therefore there is no need to use the algorithm to train the data.

6. Why does “with torch.no_grad():“ do inside the “choose_action” function in DQN?

A: The use of "with torch.no_grad():" inside the "choose_action" function in DQN disables gradient calculation for every tensor, meaning that requires_grad is set to False. This is because the function is used to choose an action and update the Q-values, but the gradients are not needed for these operations and can be disabled to improve computational efficiency.